UCAM

Tema 3

Programación con Jetpack Compose

Aplicaciones para Dispositivos Móviles

Daniel Valero Carreras

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. Navegación
2. Base de datos con Room y SQLite
3. MVVM y DI
4. Base de datos con Firebase

1. Navegación

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

1. Navegación
Navigation Compose

- Navigation Compose es la librería oficial de Jetpack para manejar la navegación entre pantallas (composables) de forma declarativa.
  Permite moverte entre rutas (screens) sin usar Fragments ni Activities adicionales.
- Se basa en tres conceptos principales:
  - NavController → controla la navegación y el historial.
  - NavHost → contenedor que muestra la pantalla activa.
  - Composable destinations → cada pantalla que se puede mostrar.

1. Navegación
Navigation Compose

- Necesitamos incorporar la librería de Navigation Compose
  - https://mvnrepository.com/artifact/androidx.navigation/navigation-compose
- Para pasarle objetos serializables también añadimos
  - https://mvnrepository.com/artifact/org.jetbrains.kotlinx/kotlinx-serialization-json-jvm

1. Navegación
Navigation Compose

- Vamos a nuestro **libs.version.toml** y añadimos en versions:
  - navigationCompose = “2.9.5”
  - kotlinSerialization = “1.8.0”
- En libraries añadimos:
  - androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }
    kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-json", version.ref = "kotlinSerialization" }
- En plugins añadimos el kotlin serialization
  - kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin" }

1. Navegación
Navigation Compose

- Vamos a nuestro **build.gradle.kts** y añadimos en plugins:
  - alias(*libs.plugins.kotlin.serialization*)
- En dependecies añadimos:
  - *implementation*(*libs.androidx.navigation.compose*)
    implementation(*libs.kotlinx.serialization.json*)
- Finalmente, sincronizamos el proyecto

1. Navegación
Screens

- Para ordenar mejor el código me creo un repositorio Screens en la carpeta ui
- Ahí coloco:
  - Un fichero screens.kt con objetos serializables que harán rereferencia a las vistas
  - Un fichero con la HomeScreen
  - Un fichero con la LoginScreen
  - …

1. Navegación

Screen.kt

```
import kotlinx.serialization.Serializable

@Serializable
object Login

@Serializable
object Home
```

1. Navegación
Navigation Wrapper

- Un **NavigationWrapper** es una **función o composable personalizado** que sirve como **envoltorio** (*wrapper*) para la navegación de la app.
- Su propósito es **organizar y simplificar** el uso del NavController, el NavHost, y las rutas de las pantallas.
- Se usa para:
  - Mantener el código ordenado y modular.
  - Evitar repetir la configuración del NavHost en diferentes lugares.
  - Tener un único punto de entrada para toda la navegación.
  - Integrar fácilmente AppBar, BottomBar, o Drawer en todas las pantallas.

1. Navegación

Navigation Wrapper

```
@Composable
fun NavigationWrapper() {
   val navController = rememberNavController() //Indica a que componente de navegación se va a
dirigir
   NavHost(navController = navController, startDestination = Login) {
      //Aquí van las rutas de navegación
      composable<Home> {
         HomeScreen({navController.navigate(Login)})

      }
      composable<Login> {
         LoginScreen({navController.navigate(Home)})
      }
   }
}
```

1. Navegación
Navigation Wrapper

- En el ejemplo el NavHost mapea hacia dónde se va en la aplicación
- El navController indica hacia donde se va en cada momento
- En cada screen, pasamos una función lambda de navegación que utilizaremos después en las screens
    - navController.navigate()-> ir a otra pantalla
    - navController.popBackStack()-> ir atrás una pantalla
    - navController.popBackStack("route", inclusive=false)-> volver hasta route

1. Navegación

HomeScreen/LoginScreen…

- Contienen la vista de la screen
- Como parámetro reciben una función lambda para que cuando se clique sobre el botón redirija a otra screen

1. Navegación

HomeScreen/LoginScreen…

```
@Composable
fun HomeScreen(nagivateToLogin: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment =
Alignment.CenterHorizontally,
        verticalArrangement =
Arrangement.Center
    ) {
        Text("Welcome to the Home Screen!")
        Button(onClick = { nagivateToLogin() }) {
Text("Ir a Login") }
    }
}
```

```
@Composable
fun LoginScreen(navigateToHome: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment =
Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("Welcome to the Login Screen!")
        Button(onClick = { navigateToHome() }) {
Text("Ir a Home") }
    }
}
```

1. Navegación

Paso de parámetros primitivos

- Al navegar se pueden pasar parámetros
  - Ints, string, etc.
- Se pueden pasar objetos pero es más complejo y NO recomendable.
- Para pasar parámetros primitivos:
  1. Creamos una data class

```
@Serializable
data class Detail(val usuario: String)
```

1. Navegación

Paso de parámetros primitivos

2. En la screen que reciba el parámetro lo tendremos que indicar

```
@Composable
fun DetailScreen(usuario: String, navigateToHome: () -> Unit) {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("Welcome to the Detail Screen " + usuario, fontSize = 24.sp)
        Button(onClick = { navigateToHome() }) { Text("Ir a Home") }
    }
}
```

1. Navegación

Paso de parámetros primitivos

3. La screen que envía debe enviar el parámetro en el navigate

```
@Composable
fun LoginScreen(navigateToHome: () -> Unit, navigateToDetail : (String) -> Unit) {
   var username by remember {mutableStateOf("")}
   Column(
      modifier = Modifier.fillMaxSize(),
      horizontalAlignment = Alignment.CenterHorizontally,
      verticalArrangement = Arrangement.Center
   ) {
      Text("Welcome to the Login Screen!")
      TextField(value = username, onValueChange = { username = it}, label = { Text(text
= "Usuario") })
      Button(onClick = { navigateToHome() }) { Text("Ir a Home") }
      Button(onClick = { navigateToDetail(username) }) { Text("Ir a detalles") }}}
```

1. Navegación

Paso de parámetros primitivos

4. En el navigation Wrapper:
    - Debemos indicar que la ruta emisora envía el objeto
    - Debemos indicar que la ruta receptora recibe una entrada
        - La función toRoute() recibe los argumentos de la ruta, los deserializa a la data class
        - Después podemos utilizar ese objeto

1. Navegación

Paso de parámetros primitivos

```
@Composable
fun NavigationWrapper() {
   val navController = rememberNavController()
   NavHost(navController = navController, startDestination = Login) {
      composable<Login> {
         LoginScreen({navController.navigate(Home)},
                  {navController.navigate(Detail(usuario = it))} )
      }
      composable<Detail> { entry ->
         val detail : Detail = entry.toRoute()
         DetailScreen( detail.usuario , {navController.navigate(Home)} )
      }}}
```

2. Base de datos con Room y SQLite

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Base de datos con Room y SQLite

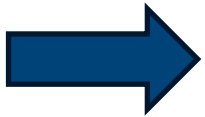
¿Por qué una base de datos local?

- Guarda datos offline
- Menos llamadas a la red → mejor rendimiento
- Persistir información del usuario:
  - Configuración
  - Historial
  - …
- Alternativa a:
  - SharedPreferences (demasiado simple)
  - Ficheros (sin esquema ni consultas)

2. Base de datos con Room y SQLite

¿Qué es SQLite?

- Motor de base de datos relacional embebido
- Viene integrado en Android
- Soporta:
  - Tablas, relaciones, índices
  - Consultas SQL (SELECT, INSERT, UPDATE, DELETE)
- Uso tradicional:
  - SQLiteOpenHelper
  - Ejecutar SQL como strings

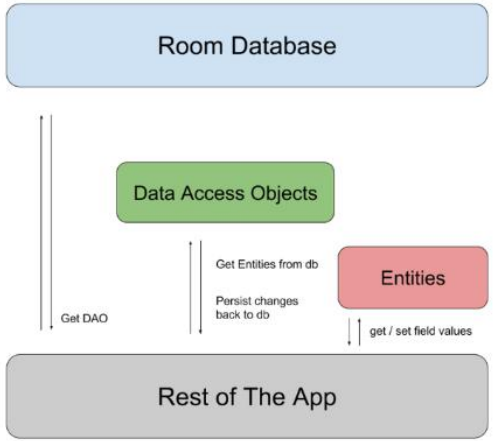


Mucho código y posibilidad de errores

2. Base de datos con Room y SQLite

¿Qué es Room?

- Librería de persistencia de Android (parte de Android Jetpack)
- Capa de abstracción sobre SQLite
- Ventajas:
  - Comprobaciones en **tiempo de compilación**
  - Anotaciones simples en lugar de SQL “a mano”
- Se basa en 3 componentes:
  - Entity: representa una tabla en la BD → data class
  - DAO (Data Access Object): interfaz con métodos para acceder a la BD
  - Database: clase abstracta que extiende de RoomDatabase y define entidades y DAOs



2. Base de datos con Room y SQLite

Añadir ROOM al proyecto

- En libs.versions.toml:
  - Versions -> room_version = "2.8.3"
  - En libraries
    - androidx-room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room_version" }
    - androidx-room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room_version" }
- En build.gradle.kts (Module app)
  - Dependencies->
    - *implementation*(*libs.androidx.room.runtime*)
    - *kapt*(*libs.androidx.room.compiler*)

2. Base de datos con Room y SQLite

Entity

- Es la tabla en base de datos
  - Lleva la etiqueta @Entity y el nombre de la tabla
  - Debe llevar una @PrimaryKey que puede ser autogenerada

```
@Entity(tableName = "ciclistas")
data class CiclistaEntity(
   @PrimaryKey(autoGenerate = true) val id:
Long = 0,
   val nombre: String,
   val equipo: String,
   val montaña: Int,
   val sprint: Int
)
```

2. Base de datos con Room y SQLite
DAO

- Separa la capa de persistencia del resto de la aplicación proporcionando una interfaz abstracta
- Proporciona métodos para consultar, insertar, borrar y actualizar datos en la BDD
  - Se pueden definir consultas con la etiqueta @Query
  - @Update, @Delete, @Insert



2. Base de datos con Room y SQLite

Ejemplo DAO

```
@Dao
interface CiclistaDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCiclista(Ciclista: CiclistaEntity) : Long

    @Query("SELECT * FROM ciclistas")
    fun getAllCiclistas(): Flow<List<CiclistaEntity>>

    @Query("SELECT * FROM ciclistas WHERE id = :id LIMIT 1")
    suspend fun getCiclistaById(id: Long): CiclistaEntity?
}
```

2. Base de datos con Room y SQLite

CiclistaMappers

- Como tenemos una clase Ciclista (UI) y una clase CiclistaEntity (Room) podemos hacer funciones mapper que convierte una a otra
  - asExternal: convierte una entidad a dominio
  - asEntity: convierte un dominio a entidad

```
fun CiclistaEntity.asExternalModel(): Ciclista =
    Ciclista(
        id = id,
        nombre = this.nombre,
        montaña = this.montaña,
        equipo = this.equipo,
        sprint = this.sprint
    )

// Dominio  ->  Entity
fun Ciclista.asEntity(): CiclistaEntity = CiclistaEntity(
    id = this.id,
    nombre = this.nombre,
    equipo = this.equipo,
    montaña = this.montaña,
    sprint = this.sprint
)
```

2. Base de datos con Room y SQLite

Clase Database

- Proporciona a tu app instancias de los DAO que definas
- Con los DAO accedes a las entitys
- Es una clase abstracta con la etiqueta @Database y los siguientes argumentos:
  - entities-> entidades que forman la base de datos
  - Version->versión de la base de datos
  - exportSchema->si quieres conservar copias de seguridad del historial de versiones
- Dentro se pone la declaración abstracta de los DAO

2. Base de datos con Room y SQLite

Clase Database

- Dentro se pone la declaración abstracta de los DAO
- Creamos un companion object()-> permite el acceso a los métodos para crear u obtener la base de datos
    - Dentro irá la función para crear la instancia de nuestra base de datos
- El código siguiente se puede utilizar de plantilla para crear la base de datos de cualquier proyecto

2. Base de datos con Room y SQLite

Ejemplo Database

```
@Database(
    entities = [CiclistaEntity::class],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun ciclistaDao(): CiclistaDao
    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null
        fun getInstance(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "ciclistas_db"
                ).build().also { INSTANCE = it }
            }
        }
    }
}
```

2. Base de datos con Room y SQLite

Uso Database

- Crearíamos un objeto DAO a través de la instancia a la base de datos.
- Podríamos persistir en datos lanzando una corrutina->scope.launch

- ESTO FUNCIONA pero no es lo recomendado por el estándar
  - LA UI no debe actuar con ENTITIES sino con objetos de dominio

```
fun LoginScreen(navigateToHome: (Long) -> Unit = {}) {

  ...
   val scope = rememberCoroutineScope()
   val context = LocalContext.current
   val ciclistaDao = remember {
      AppDatabase.getInstance(context).ciclistaDao()
   }

  ...
Button(onClick = {
   val ciclista = ...
   )
   scope.launch{
      val idCiclista = ciclistaDao.insertCiclista(ciclista.asEntity())
      Log.i("LoginScreen", "Ciclista creado con ID: ${idCiclista}")
      navigateToHome(idCiclista)
   }
...
```

2. Base de datos con Room y SQLite

Vista base de datos en Android Studio

- Crearíamos un objeto DAO a través de la instancia a la base de datos.
- Podríamos persistir en datos lanzando una corrutina->scope.launch

- ESTO FUNCIONA pero no es lo recomendado por el estándar
  - LA UI no debe actuar con ENTITIES sino con objetos de dominio

```
fun LoginScreen(navigateToHome: (Long) -> Unit = {}) {

  ...
   val scope = rememberCoroutineScope()
   val context = LocalContext.current
   val ciclistaDao = remember {
      AppDatabase.getInstance(context).ciclistaDao()
   }

  ...
Button(onClick = {
   val ciclista = ...
   )
   scope.launch{
      val idCiclista = ciclistaDao.insertCiclista(ciclista.asEntity())
      Log.i("LoginScreen", "Ciclista creado con ID: ${idCiclista}")
      navigateToHome(idCiclista)
   }
...
```

3. MVVM y DI

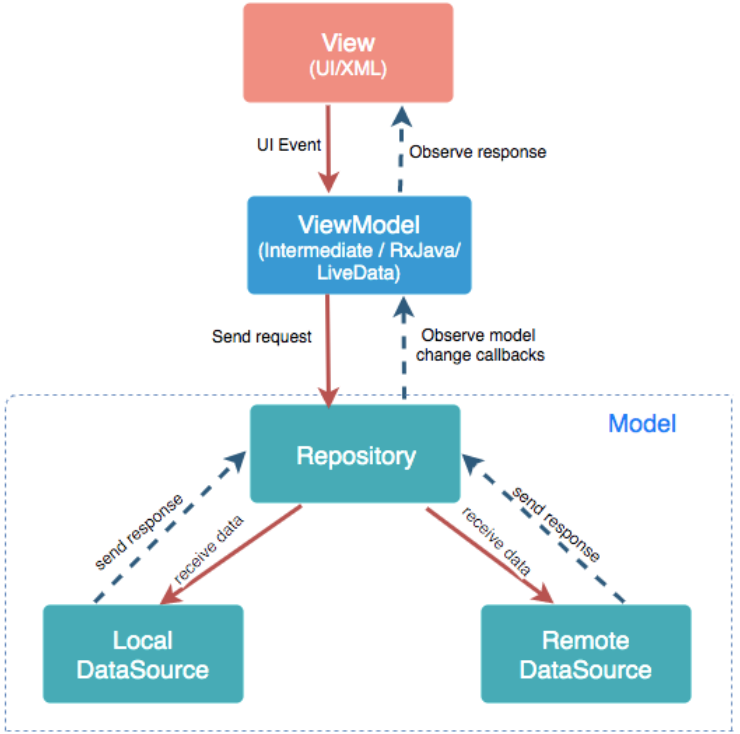
Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

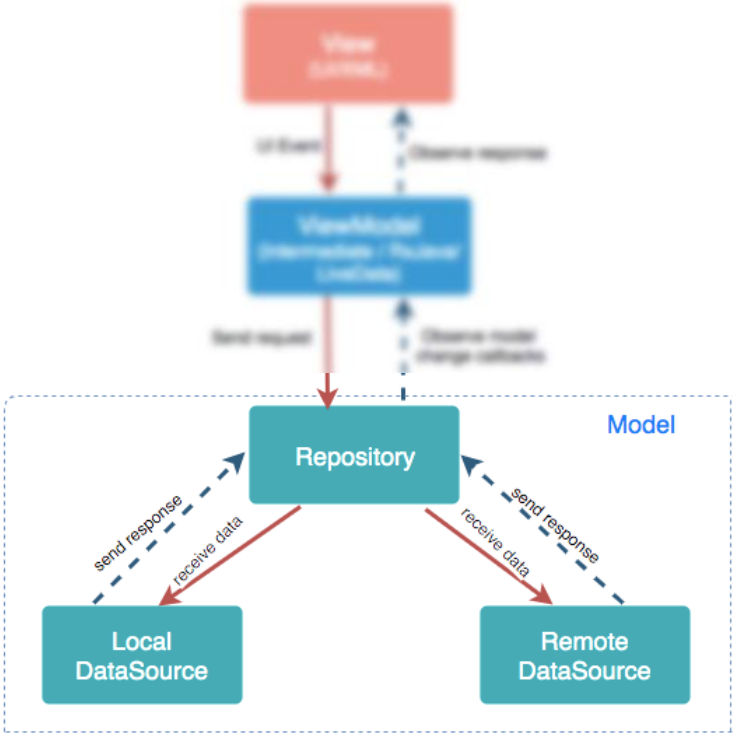
3. MVVM y DI
Introducción

- MVVM (Model–View–ViewModel) es un **patrón de arquitectura** para organizar el código de una app, separando claramente:
  - la **lógica de negocio y datos**
  - de la **lógica de presentación**
  - y de la **interfaz de usuario**
- De esta forma el código es más **mantenible, testeable y reutilizable**.



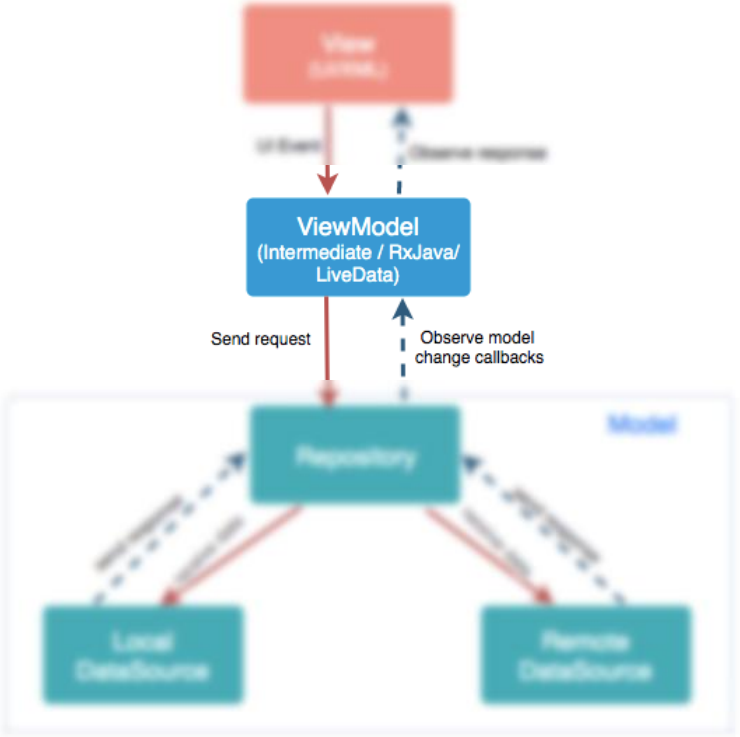
3. MVVM y DI
Introducción

- **Model:**
  - Datos y reglas de negocio
  - Formado por:
    - Entidades
    - Repositorios
    - Fuentes de datos
  - No sabe nada de la UI



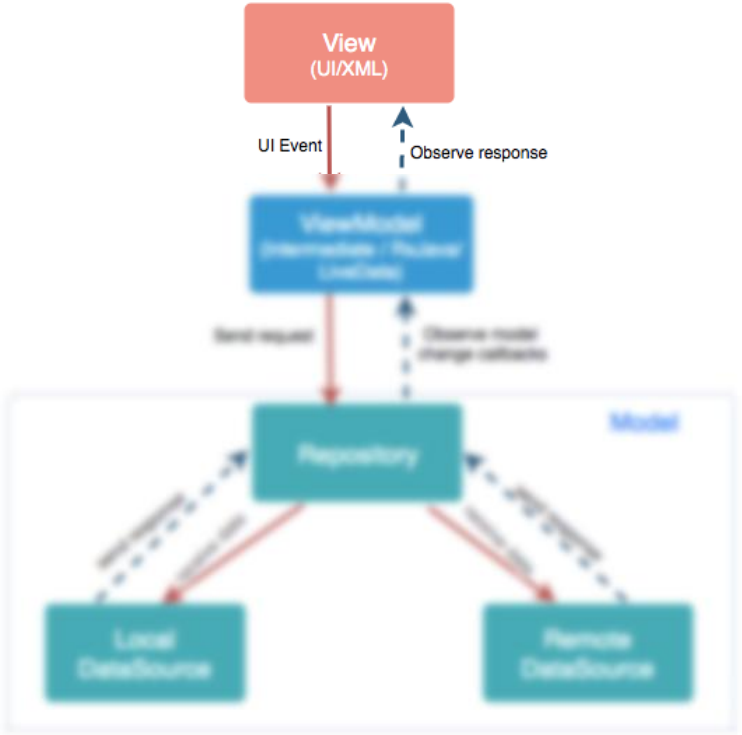
3. MVVM y DI
Introducción

- **ViewModel:**
- Expone **estado observable** a la View
- Recibir las **acciones de la UI** y traducirlas en llamadas al Model.



3. MVVM y DI
Introducción

- **View:**
- Es la **interfaz de usuario**: **Composables**
- Se encarga de:
  - Dibujar la UI en función de un **estado**.
  - Enviar las **acciones del usuario** (clicks, inputs) al ViewModel.
- La View **no debería tener lógica de negocio**; únicamente lógica de presentación simple.



3. MVVM y DI
Repository

- **Esconde** de la app cómo se guardan/leen datos (Room, red, caché…).
- Te devuelve **modelos de dominio** (Ciclista) y no entities de Room.
- Es el “sitio único” donde se decide reglas de datos.
- Con Repositories:
  - La UI no ve entities
  - No sabe que existe ROOM
  - Si cambio ROOM por API la UI le da igual

```
class CiclistaRepository(private val dao: CiclistaDao) {

    val ciclistas = dao.getAllCiclistas()
        .map { list -> list.map { it.asExternalModel() } }

    suspend fun insert(ciclista: Ciclista): Long =
        dao.insertCiclista(ciclista.asEntity())

    suspend fun getById(id: Long): Ciclista? =
        dao.getCiclistaById(id)?.asExternalModel()
}
```

3. MVVM y DI
ViewModel

- Es una clase que mantiene el estado de la pantalla
- Guarda los estados de la UI y la sobrevive a rotaciones, etc.
- Conecta con los repositories que a su vez conectan con los DAOs para acceder a datos

```
class CiclistaViewModel(private val repo: CiclistaRepository) : ViewModel()
{

    val ciclistas = repo.ciclistas
        .stateIn(viewModelScope,
SharingStarted.Companion.WhileSubscribed(5_000), emptyList())

    private val _ciclistaActual = MutableStateFlow<Ciclista?>(null)
    val ciclistaActual: StateFlow<Ciclista?> = _ciclistaActual

    fun cargarCiclista(id: Long) {
        viewModelScope.launch {
            _ciclistaActual.value = repo.getById(id)
        }
    }

    fun insertarYDevolverId(ciclista: Ciclista, onId: (Long) -> Unit) {
        viewModelScope.launch {
            val id = repo.insert(ciclista)
            onId(id)
        }
    }
}
```

3. MVVM y DI
Ejemplo ViewModel

- En el repository repo.ciclistas devuelve todos los ciclistas (getAll)… Aquí val ciclistas -> convierte un Flow en un StateFlow
  - sharing: se mantiene activo 5 segundos si alguien lo observa

```
class CiclistaViewModel(private val repo: CiclistaRepository) : ViewModel()
{

    val ciclistas = repo.ciclistas
        .stateIn(viewModelScope,
SharingStarted.Companion.WhileSubscribed(5_000), emptyList())

    private val _ciclistaActual = MutableStateFlow<Ciclista?>(null)
    val ciclistaActual: StateFlow<Ciclista?> = _ciclistaActual

    fun cargarCiclista(id: Long) {
        viewModelScope.launch {
            _ciclistaActual.value = repo.getById(id)
        }
    }

    fun insertarYDevolverId(ciclista: Ciclista, onId: (Long) -> Unit) {
        viewModelScope.launch {
            val id = repo.insert(ciclista)
            onId(id)
        }
    }
}
```

3. MVVM y DI

Inyección de dependencias (DI)

- La inyección de dependencias se hace para evitar que cada screen o cada parte de la aplicación tenga que estar recibiendo ViewModels
- Para ello se utilizar Hilt: librería de inserción de dependencias de Dagger
- Necesarios:
  - Libraries:
    dagger-hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "dagger_version" }
    dagger-hilt-compiler = { group = "com.google.dagger", name = "hilt-compiler", version.ref = "dagger_version" }
    androidx-hilt-navigation-compose = { group = "androidx.hilt", name = "hilt-navigation-compose", version.ref = "hiltNavigationCompose" }

3. MVVM y DI

Inyección de dependencias (DI)

- Necesarios:
  - Plugins:
    hilt-android = { id = "com.google.dagger.hilt.android", version.ref = "dagger_version" }
    ksp = { id = "com.google.devtools.ksp", versión.ref = “ksp_version" }
- Añadir los plugins a build.gradle de Project y del Module
- Añadir las dependencias a buil.gradle del Module

3. MVVM y DI

Inyección de dependencias (DI)

- Todas las clases que utilizan Hilt necesitan una clase Application anotada con @HiltAndroid
- Hay que añadir esta clase manifest:
- <application
  android:name=".CyclingApplication"
- …
- </application>

```
@HiltAndroidApp
class CyclingApplication: Application()
```

3. MVVM y DI
Inyección de dependencias (DI)

- Una vez que se configura Hilt en tu clase Application y hay un componente disponible en el nivel de la aplicación, Hilt puede proporcionar dependencias para otras clases de Android que tengan la anotación @AndroidEntryPoint

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
   override fun onCreate(savedInstanceState:
Bundle?) {
      super.onCreate(savedInstanceState)
      enableEdgeToEdge()
      setContent {
         CyclingAppTheme {
            NavigationWrapper()
         }
      }
   }
}
```

3. MVVM y DI

Inyección de dependencias (DI)

- Creamos un modulo DI para Room+repository
- De este modo tenemos una instancia única de la base de datos, del DAO y del Repository
- Ahora se crea aquí la BBDD aunque se define en la otra clase

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {
  @Provides
  @Singleton
  fun provideDatabase(
    @ApplicationContext context: Context
  ): AppDatabase =
    Room.databaseBuilder(
      context,
      AppDatabase::class.java,
      "ciclistas_db"
    )
      .build()
  @Provides
  fun provideCiclistaDao(db: AppDatabase): CiclistaDao =
    db.ciclistaDao()
  @Provides
  @Singleton
  fun provideCiclistaRepository(dao: CiclistaDao): CiclistaRepository =
    CiclistaRepository(dao)
}
```

3. MVVM y DI

Inyección de dependencias (DI)

- Podemos cambiar nuestra AppDatabase que habíamos creado antes.

```
@Database(
    entities = [CiclistaEntity::class],
    version = 2,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun ciclistaDao(): CiclistaDao

}
```

3. MVVM y DI

Inyección de dependencias (DI)

- El viewModel debe llevar ahora la etiqueta de @HiltViewModel
- Hilt sabe cómo construir el repo, porque AppModule lo explica.

```
@HiltViewModel
class CiclistaViewModel @Inject constructor(
    private val repo: CiclistaRepository
) : ViewModel() {
    val ciclistas = repo.ciclistas
        .stateIn(viewModelScope,
            SharingStarted.WhileSubscribed(5_000),
            emptyList())
    private val _ciclistaActual = MutableStateFlow<Ciclista?>(null)
    val ciclistaActual: StateFlow<Ciclista?> = _ciclistaActual
    fun cargarCiclista(id: Long) {
        viewModelScope.launch {
            _ciclistaActual.value = repo.getById(id)
        }
    }
    fun crearCiclistaYDevolverId(ciclista: Ciclista, onId: (Long) -> Unit) {
        viewModelScope.launch {
            val id = repo.insert(ciclista)
            onId(id)
        }
    }
    fun borrarCiclista(ciclista: Ciclista) {
        viewModelScope.launch {
            repo.delete(ciclista)
        }
    }
}
```

3. MVVM y DI
Inyección de dependencias (DI)

- Ya tenemos inyectado nuestro ViewModel y ahora podemos acceder a él desde cualquier sitio con hiltViewModel()

```
fun LoginScreen(navigateToHome: (Long) -> Unit = {}) {
  val viewModel: CiclistaViewModel = hiltViewModel()

Button(onClick = {
   val ciclista = Ciclista(
      nombre = textNombre,
      montaña = montaña,
      equipo = equipo,
      sprint = sprint
   )
   viewModel.crearCiclistaYDevolverId(ciclista) { idCiclista ->
      Log.i("LoginScreen", "Ciclista creado con ID: $idCiclista")
      navigateToHome(idCiclista)
   }

}) {
   Text(text = "Iniciar")
}
```

4. Base de datos con Firebase

Imagen 1
Título asignado a la imagen

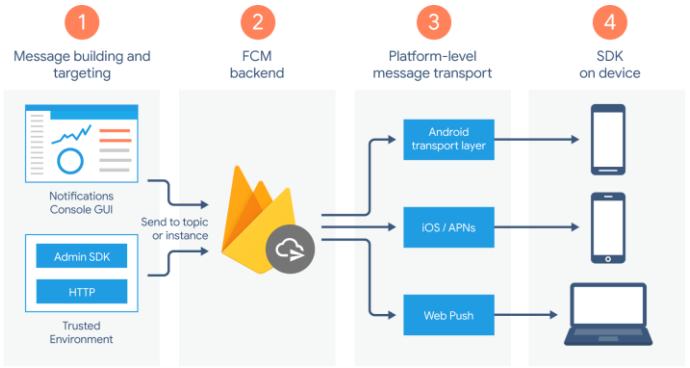


Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

4. Base de datos con Firebase

¿Qué es Firebase?

- Firebase (Firestore / Realtime Database)
- Base de datos en la nube gestionada por Google
- Sin necesidad de crear y mantener un servidor propio
- Acceso mediante SDK oficial de Android (Kotlin/Java)
- Datos en tiempo real: se actualizan solos en la UI
-



4. Base de datos con Firebase

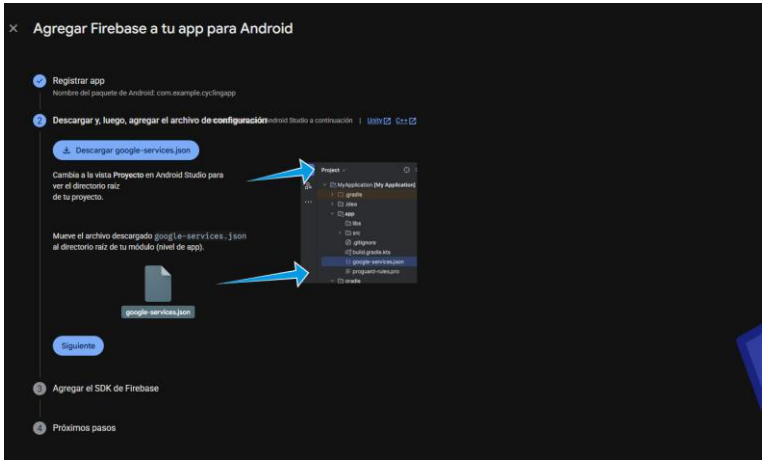
¿Por qué usar Firebase como base de datos en Android?

- Desarrollo de apps modernas
- Datos en la nube y en tiempo real
- Menos backend, más foco en la app
- Supera limitaciones de SQLite
  - Los datos se guardan solo en el dispositivo
  - No es trivial sincronizar entre varios móviles
  - Copias de seguridad, migraciones y seguridad dependen de ti
  - Si el usuario borra la app → se pierden los datos

4. Base de datos con Firebase

Adaptar proyecto Room a Firebase

- Accedemos a https://console.firebase.google.com/
- Creamos un proyecto
- Cuando se haya creado le damos a agregar app-> Android
- Le damos el nombre de nuestro paquete y registramos la app
- Descargamos el fichero Google-services.json y lo añadimos al proyecto
- Agregamos las dependencias y plugins



4. Base de datos con Firebase

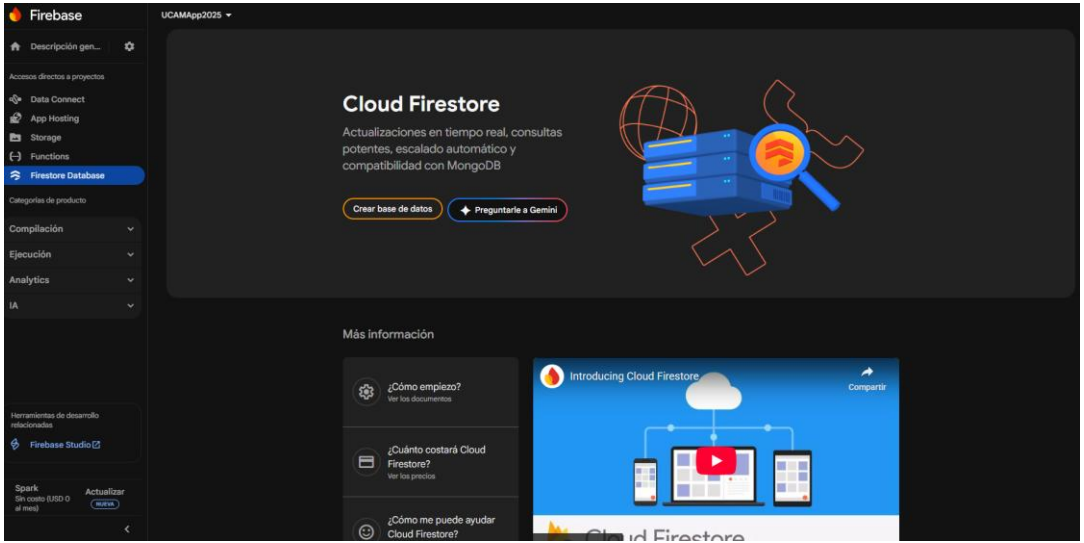
Adaptar proyecto Room a Firebase

- Plugins (Project)
  - id("com.google.gms.google-services") version "4.4.4" apply false
- Plugins (app)
  - id("com.google.gms.google-services")
- Dependencies (app)
  - Firebase-bom
  - Firebase-analytics
  - Firebase-firestore-ktx

4. Base de datos con Firebase

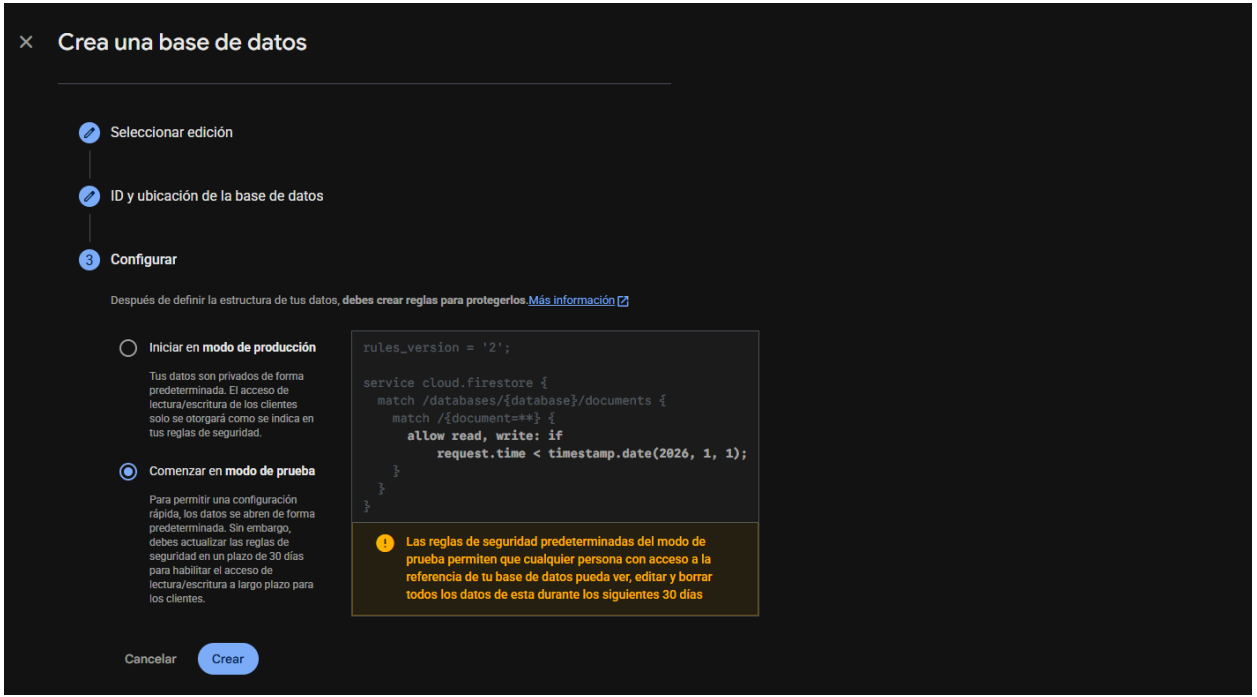
Acceso a consola

- Accedemos a la consola de Firebase
- Vamos a Firestore Database y Crear base de datos



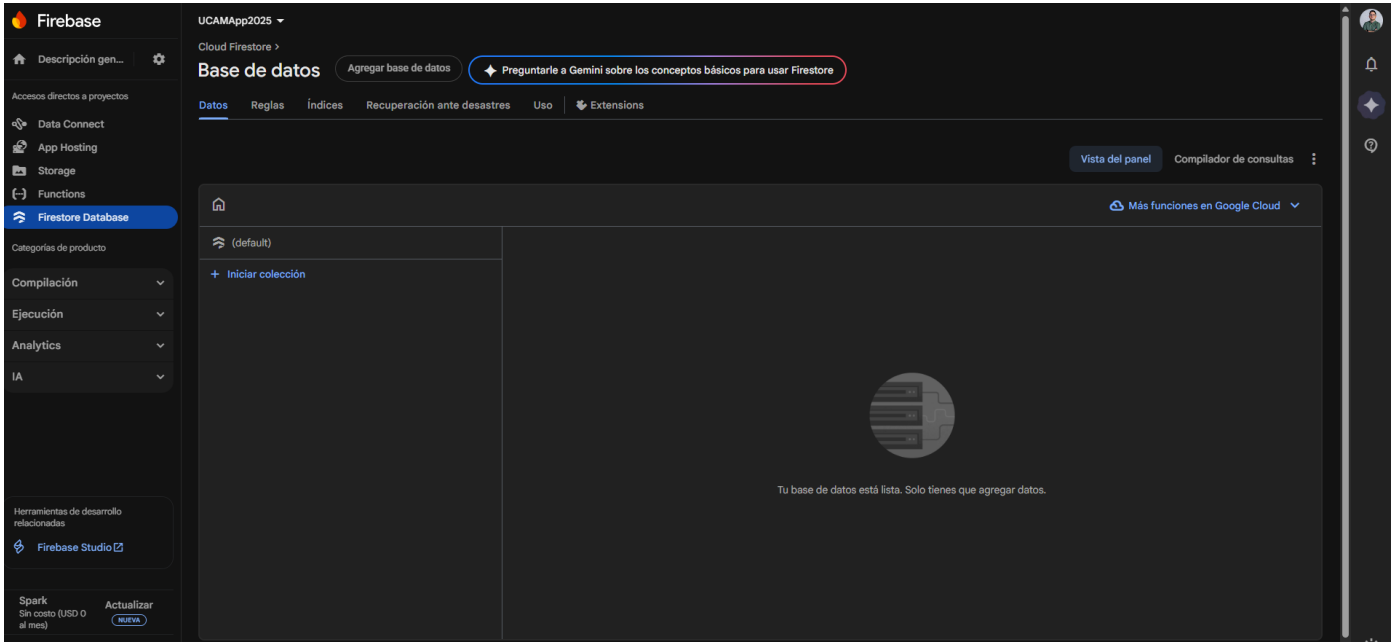
4. Base de datos con Firebase

Acceso a consola



4. Base de datos con Firebase

Acceso a consola



4. Base de datos con Firebase
Repository

- En este caso no recibe DAOs
- Recibe la conexión Firebase
- Para acceder a una colección-> collection
  - callBackFlow -> devuelve llamadas a listeners en un Flow (flujo de datos)
  - AddSnapshotListener -> escucha cambios en la colección en tiempo real
  - snapshot-> estado de la colección actualmente

```
class CiclistaRepository @Inject constructor(
    private val firestore: FirebaseFirestore
) {
    private val collection = firestore.collection("ciclistas")

        val ciclistas: Flow<List<Ciclista>> = callbackFlow {
        val listener = collection.addSnapshotListener { snapshot, error ->
            if (error != null) {
                close(error)
                return@addSnapshotListener
            }
            val lista = snapshot?.documents?.mapNotNull { doc ->
                doc.toObject<Ciclista>()
            } ?: emptyList()
            trySend(lista)
        }

        awaitClose { listener.remove() }
    }
}
```

4. Base de datos con Firebase

Repository

- trySend(lista)-> envía la lista al Flow así cualquiera que esté haciendo un collect de ciclista recibe la lista de ciclistas
- awaitClose{listener.remove()}-> cancela la recolección del Flow
- EN RESUMEN:
  - Genera un flujo que escucha cambios en la colección y lo convierte en lista para que sea utilizada

```
class CiclistaRepository @Inject constructor(
    private val firestore: FirebaseFirestore
) {
    private val collection = firestore.collection("ciclistas")

        val ciclistas: Flow<List<Ciclista>> = callbackFlow {
        val listener = collection.addSnapshotListener { snapshot, error ->
            if (error != null) {
                close(error)
                return@addSnapshotListener
            }
            val lista = snapshot?.documents?.mapNotNull { doc ->
                doc.toObject<Ciclista>()
            } ?: emptyList()
            trySend(lista)
        }

        awaitClose { listener.remove() }
    }
}
```

4. Base de datos con Firebase

CRUD con Firebase

- Collection.document.set()-> insert
- Collection.document.get()->query
- Collection.documente.delete()->delete

```
class CiclistaRepository @Inject constructor(
    private val firestore: FirebaseFirestore
) {
suspend fun insert(ciclista: Ciclista): Long {
    // Creamos un id basado en tiempo
    val idGenerado = System.currentTimeMillis()
    val ciclistaConId = ciclista.copy(id = idGenerado)

    collection.document(idGenerado.toString()).set(ciclistaConId).await()
    return idGenerado
}

suspend fun getById(id: Long): Ciclista? {
    val doc = collection.document(id.toString()).get().await()
    return doc.toObject<Ciclista>()
}

suspend fun delete(ciclista: Ciclista) {
    collection.document(ciclista.id.toString()).delete().await()
}


}
```

4. Base de datos con Firebase

Inyección de dependencias (DI)

- En el AppModule
  - Crear un provider de la base de datos de Firebase
  - Repository de ciclista

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideFirestore(): FirebaseFirestore = Firebase.firestore

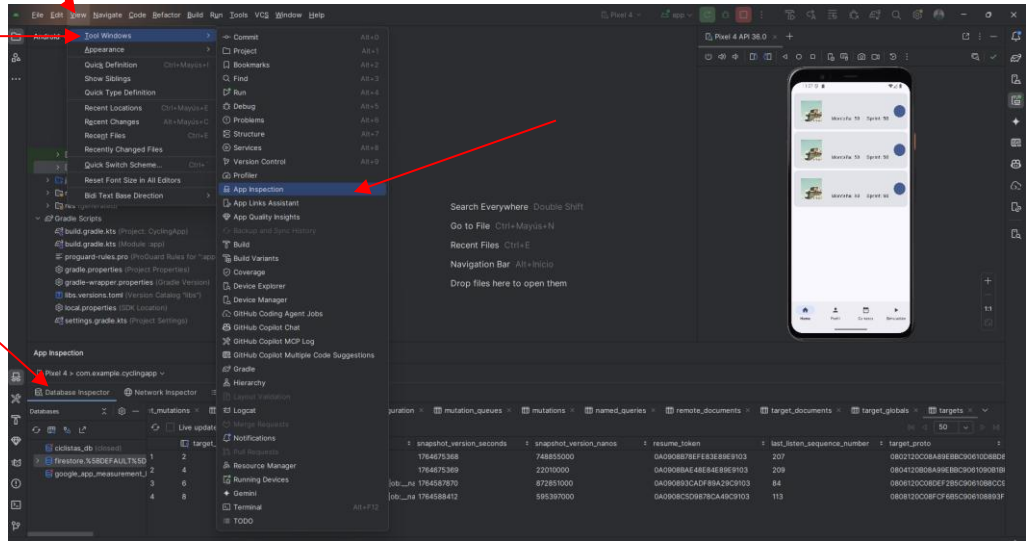
    @Provides
    @Singleton
    fun provideCiclistaRepository(
        firestore: FirebaseFirestore
    ): CiclistaRepository = CiclistaRepository(firestore)

    @Provides
    @Singleton
    fun provideCarreraRepository(
        firestore: FirebaseFirestore
    ): CarreraRepository = CarreraRepository(firestore)
}
```

4. Base de datos con Firebase

Visualizar las BBDD en Android Studio

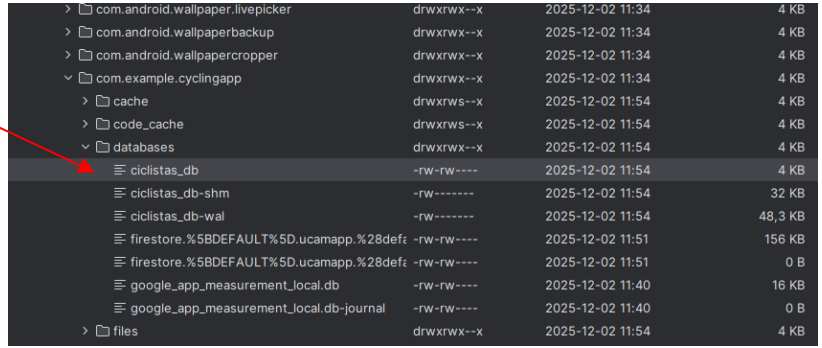
- Podemos visualizar la base de datos SQLite de nuestra aplicación
- Para ello vamos a View->Tool Windows-> App Inspection
- Despues aparecerá DatabaseInspector-> ahí se visualiza nuestra base de datos

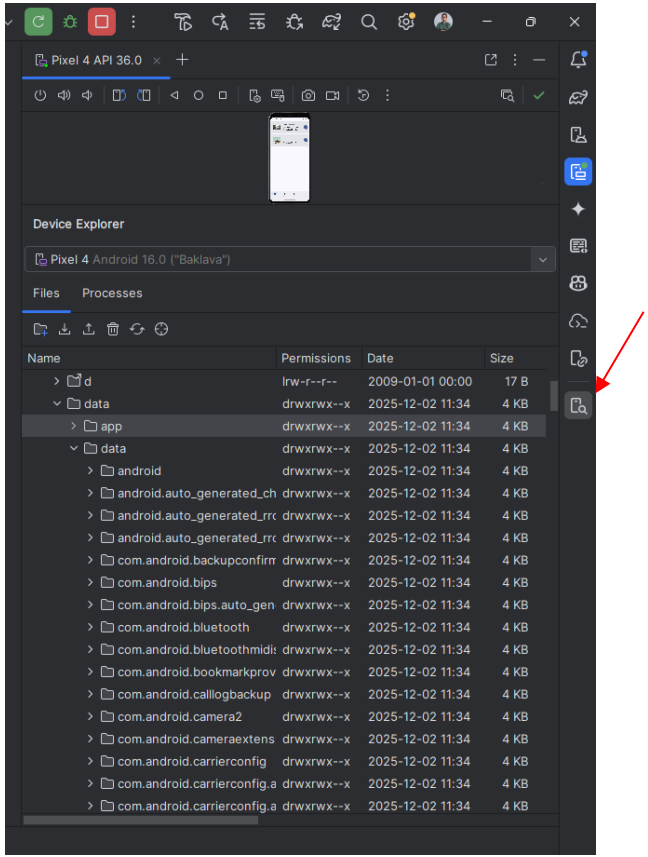


4. Base de datos con Firebase

Visualizar las BBDD en Android Studio

- En el Device Explorer podemos ver todos los ficheros de nuestra MV de Android
- En la carpeta: data/data/com.example.nombre_app/databases se almacen nuestra base de datos de SQLite.





REFERENCIAS

BIBLIOGRÁFICAS

OpenCompose: https://www.jetpackcompose.pro/

Cómo conservar datos con Room: https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?utm_source=chatgpt.com&hl=es-419#0

Inserción de dependencias con Hilt: https://developer.android.com/training/dependency-injection/hilt-android?hl=es-419

Agrega Firebase al proyecto de Android: https://firebase.google.com/docs/android/setup?hl=es-419

Daniel Valero Carreras

dvalero@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM